

CS5224: Cloud Computing
AY25/26, Y3S1
Notes

Sim Ray En Ryan

December 14, 2025

Contents

1	L01: Introduction	5
1.1	Three Eras of Computing	5
1.1.1	Cognitive Computing	5
1.2	What is Cloud Computing?	5
1.2.1	Definitions	5
1.2.2	Key Defining Attributes (NIST)	5
1.2.3	Enabling Technologies	6
1.3	Why Cloud Computing?	6
1.3.1	Mismatch between Demand and Capacity (Traditional IT)	6
1.4	Key Business Drivers	7
1.5	Basic Concepts and Terminology	7
1.5.1	Key Terms	7
1.5.2	Scaling	7
1.5.3	Cloud Service	8
1.5.4	Trust Boundary	8
1.6	Technical and Non-Technical Challenges	8
1.6.1	Technical Challenges	8
1.6.2	Non-Technical Challenges	9
2	L02: Concepts and Models	9
2.1	Cloud Computing (NIST 2011)	9
2.1.1	5 Essential Characteristics	9
2.1.2	Resource Pooling (Multi-tenancy)	9
2.2	Cloud Service (Delivery) Models	10
2.2.1	Why different service models?	10
2.2.2	Comparison of Responsibilities (Consumer vs. Provider)	10
2.2.3	Pros and Cons	11
2.3	Conceptual Reference Architecture (NIST 2011)	11
2.3.1	5 Major Actors	11
2.3.2	Cloud Provider Functions (Service Orchestration and Management)	11
2.4	Cloud Deployment Models	12
3	L03: Cloud Architecture	12
3.1	How to Organize (Partition) Resources?	12
3.1.1	1. Workload Distribution Architecture	12
3.1.2	2. Resource Pooling Architecture	13
3.1.3	3. Dynamic Scalability Architecture (Demand Fluctuations)	13
3.1.4	4. Elastic Resource Capacity Architecture (Resource Availability Fluctuations)	13
3.1.5	5. Service Load Balancing Architecture	14
3.2	How to Operate/Manage Resources to Meet Certain Objectives?	14

3.2.1	Cloud Bursting Architecture	14
3.2.2	Elastic Disk Provisioning Architecture	14
4	L04: Resource Hosting & Datacenter	15
4.1	Resource Hosting	15
4.1.1	Latency	15
4.1.2	Bandwidth	15
4.2	Main Components in a Datacenter	15
4.2.1	Hardware Components	15
4.2.2	Infrastructure and Protection	16
4.3	Datacenter Availability and Efficiency	16
4.3.1	Datacenter Tiers (Increasing Availability & Cost)	16
4.3.2	Energy Usage and Power Efficiency	17
5	L05: Virtualization & Multitenancy	17
5.1	Virtualization	17
5.1.1	Pros and Cons	17
5.2	Types of Virtualization	18
5.3	Approaches to Virtualization (Hypervisors)	18
5.3.1	Virtualization Approaches	18
5.3.2	Types of Hypervisors	19
5.4	Containers (vs. VMs)	19
5.5	Multitenancy	19
5.5.1	Characteristics of Multitenancy	20
6	L06: Applications and Paradigms	20
6.1	Cloud Applications	20
6.1.1	Common Features of Cloud Providers	20
6.1.2	Ideal Applications for Cloud	20
6.1.3	Challenges in Developing Cloud Applications	20
6.1.4	Architectural Styles for Cloud Applications	21
6.2	Cloud Application Development Models	21
6.3	Function-as-a-Service (FaaS) and Serverless Computing	22
6.4	MapReduce Programming Model	22
6.4.1	Hadoop	22
7	L08: Cloud Software Development	23
7.1	SaaS vs. Traditional Software	23
7.1.1	SaaS is Different	23
7.1.2	Suitability of SaaS	23
7.2	Different Perspectives of SaaS Development	23
7.3	Cloud-aware Software Development using PaaS	24

8	L09: Pricing Models and Modeling TCO	25
8.1	Pricing Models	25
8.1.1	Factors Influencing Price Model	25
8.1.2	Key Variables	25
8.1.3	Examples of Pricing	25
8.1.4	SaaS Pricing Models	25
8.1.5	AWS Pricing Options (EC2)	26
8.2	Cost Metrics	26
8.2.1	Business Cost Metrics	26
8.2.2	Cloud Usage Cost Metrics	26
8.3	Total Cost of Ownership (TCO) of a Datacenter	27

1 L01: Introduction

1.1 Three Eras of Computing

1. Tabulating (1890s - 1940s)

- Used mechanical tabulators to count and summarize information.
- Limited to a single task.

2. Programming (1950s – present)

- Used programmable systems to perform different and multiple tasks.

3. Cognitive (2011 - future)

- Extend the boundaries of human cognition.
- Learn and adapt over time.

1.1.1 Cognitive Computing

- **Cognition:** The act of thinking.
- **Cognitive Computing:** "systems that learn at scale, reason with purpose and interact with human naturally".
- Driven by the explosion of unstructured data.
- Broad capabilities: discovery, engagement, and decision.

1.2 What is Cloud Computing?

1.2.1 Definitions

- **Gartner Report:** "... a style of computing in which scalable and elastic IT-enabled capabilities are delivered as a service to external customer using Internet technologies".
- **Forrester Research:** "... a standardized IT capability (services, software, or infrastructure) delivered via Internet technologies in a pay-per-use, self-service way."
- **NIST 2011:** "Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources... that can be rapidly provisioned and released with minimal management effort or service provider interaction." (Key: on-demand service + elastic resource)

1.2.2 Key Defining Attributes (NIST)

1. On-demand self-service

2. Broad network access
3. Resource pooling
4. Rapid elasticity
5. Measured service
6. Resiliency

1.2.3 Enabling Technologies

Cloud computing is a combination of enabling technologies to deliver computing-as-a-service:

- Parallel & distributed computing
- Programming models
- Utility computing
- Storage & network technologies
- Virtualization
- Internet (Web x.0)

1.3 Why Cloud Computing?

- Delivers cognitive services (e.g., IBM Watson, Amazon AI, Google AI).
- Provides technology for current/future innovations and disruptions (e.g., AI, blockchain, IoT).
- Reduces business cost.
- Improves the match between elastic resource demand and elastic computing resources.

1.3.1 Mismatch between Demand and Capacity (Traditional IT)

- **Elastic user demand:** Subject to peak/trough usage and business cycles.
- **Mismatch with fixed (in-house) resource capacity:**
 - **Under-provisioning:** Loss of potential revenue from users not served (customers lost).
 - **Over-provisioning:** Too much excess capacity ("opportunity cost") or huge capital expenditure (scale-up).

- **Cloud Solution:** Automated elasticity allows resource supply (the green line in the diagram) to closely follow the actual demand (the red line), minimizing waste and loss.

1.4 Key Business Drivers

1. **Capacity planning (Resource Provisioning):** Cloud enables on-demand self-service, rapid elasticity, and measured service, offering solutions to challenges like usage/demand fluctuations and provisioning costs.
2. **Cost reduction:** Cloud computing offers cost efficiency at scale by shifting from up-front capital expenditure (Capex) in traditional IT to variable operational costs (Opex) (pay-per-use model).
3. **Organizational agility:** Cloud provides elastic IT resources to quickly respond to unpredictable business cycles, allows for faster software fixes/updates, and rapid introduction of new hardware innovations via datacenters.

1.5 Basic Concepts and Terminology

1.5.1 Key Terms

1. **Elastic resource**
2. **Availability**
3. **Capacity planning** (resource provisioning)
4. **Scaling** (horizontal, vertical)
5. **Cloud-based IT resources** (servers, storage, network, applications, services)
6. **Cloud service**
7. **Trust boundary**

1.5.2 Scaling

The ability of the IT resource to handle increased or decreased usage (business) demands.

- **Horizontal scaling (out or in):** Add or remove the same resource type (e.g., adding more virtual servers).
- **Vertical scaling (up or down):** Replace a resource with higher/lower capacity or add resources to a single node (e.g., replacing a virtual server with 2 CPUs with a physical server with 4 CPUs).

Scaling Comparison

- **Horizontal:** Less expensive (commodity hardware), instantly available, easy setup (replication/auto-scaling), hardware capacity is **not** limited.
- **Vertical:** More expensive (specialized hardware), normally instantly available, may need additional setup, hardware capacity is **limited** by maximum capacity.

1.5.3 Cloud Service

Any IT resource made remotely accessible via a cloud.

- A cloud service is accessed by a consumer outside the cloud boundary.
- A multitude of service usage models: Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS), Software-as-a-Service (SaaS).

1.5.4 Trust Boundary

- **Organizational Boundary:** The physical scope of IT resources owned and governed by an organization.
- **Trust Boundary:** A logical perimeter that represents which IT resources are trusted by an organization.
- In cloud computing, the trust boundary expands to encompass the cloud service provider's environment.
- **Implications:** Increased security vulnerabilities and reduced operational governance control.
- **Overlapping Trust Boundaries:** Occurs when shared IT resources across different cloud consumers introduce overlapping boundaries, potentially allowing malicious consumers to steal or damage data.

1.6 Technical and Non-Technical Challenges

1.6.1 Technical Challenges

- Software development complexity across different cloud platforms and providers.
- Tools are continuously evolving.
- Moving large data is still expensive.
- Security concerns.
- Internet dependence (latency and bandwidth issues).
- Quality of service (QoS).

- Energy use.

1.6.2 Non-Technical Challenges

- Increased security vulnerabilities (shared responsibility with cloud providers, expansion/overlap of trust boundaries).
- Reduced operational governance control (unreliable provider/network connection affecting SLA).
- Privacy/legal issues (data localization, multi-regional compliances).
- Vendor lock-in.
- Non-standardization.
- Service level agreement (SLA) issues.

2 L02: Concepts and Models

2.1 Cloud Computing (NIST 2011)

- Defined by a "5-4-3" structure:
 - **5 Essential Characteristics**
 - **4 Deployment Models**
 - **3 Service Models**

2.1.1 5 Essential Characteristics

1. On-demand self-service (through a service portal)
2. Broad network access (ubiquitous access)
3. Resource pooling (multi-tenancy, location-independent)
4. Rapid elasticity (fast deployment/time to market)
5. Measured service (pay-per-use)

2.1.2 Resource Pooling (Multi-tenancy)

- A cloud provider pools its IT resources to serve multiple cloud service consumers (tenants).
- An instance of the program serves different consumers, where each instance is isolated from the others.

- **Single-tenant:** Each cloud consumer has a separate IT resource instance.

2.2 Cloud Service (Delivery) Models

Represented in a hierarchy based on the level of abstraction, with IaaS at the base and SaaS at the top.

1. IaaS (Infrastructure-as-a-Service)

- **What:** Rent processing, storage, network capacity, and other fundamental computing resources.
- **Product:** Virtual Machine Instances (VMs), computing resources (CPU, memory, disk) made available to users.
- **Example:** Amazon EC2.

2. PaaS (Platform-as-a-Service)

- **What:** Deploy customer-related applications to a cloud. A software platform for developers to build cloud applications.
- **Product:** Ready-made environment, application server + DBMS platforms.
- **Example:** Google's App Engine, Amazon AWS, Microsoft Windows Azure.

3. SaaS (Software-as-a-Service)

- **What:** Use the provider's applications over a network. Applications running on browsers.
- **Product:** Cloud software service.
- **Example:** Salesforce.com, Google Apps.

2.2.1 Why different service models?

Different levels of abstraction provide:

- Ease of use (setup & programming).
- Reduced management effort.
- Fast resource deployment.

2.2.2 Comparison of Responsibilities (Consumer vs. Provider)

— Component — On-premise — IaaS Provider — PaaS Provider — SaaS Provider —
 ————— Network, Storage, Server, Virtualization, OS, Database, Inte-
 gration, Runtime, Application, Data — Consumer — IaaS Provider handles Virtualization,

Server, Storage, Network. Consumer handles the rest. — PaaS Provider handles OS, Virtualization, Server, Storage, Network, Database, Integration, Runtime. Consumer handles Application, Data. — SaaS Provider handles everything from Network up to Application. Consumer handles Data. —

2.2.3 Pros and Cons

— Service Model — Pros — Cons — ————— — On-premise — Highest flexibility — Long time-to-market — — IaaS — Scalability, no hardware procurement — Privacy — — PaaS — DB, Frameworks, middleware ready — Vendor lock-in — — SaaS — Fastest time-to-market — Lowest flexibility —

2.3 Conceptual Reference Architecture (NIST 2011)

A vendor- and implementation-independent framework to describe, discuss, and develop a system-specific architecture.

2.3.1 5 Major Actors

An **actor** is an entity (person or organization) that participates in a transaction or process.

1. **Cloud Consumer:** Maintains a business relationship with, and uses services from, Cloud Providers.
2. **Cloud Provider:** Offers a cloud service to cloud consumers.
3. **Cloud Auditor:** Conducts independent assessment (audits) of cloud services, system operations, performance, and security.
4. **Cloud Broker:** Manages the use, performance, and delivery of cloud services, and negotiates relationships between Cloud Providers and Cloud Consumers.
 - **Service intermediation:** Enhances a given service (e.g., managing access, enhanced security).
 - **Service aggregation:** Combines and integrates multiple services into one or more new services.
 - **Service arbitrage:** Provides flexibility to choose services from multiple agencies (similar to aggregation but services aggregated are not fixed).
5. **Cloud Carrier:** Provides connectivity and transport of cloud services (network, telecommunication).

2.3.2 Cloud Provider Functions (Service Orchestration and Management)

Service Orchestration (3 Layers) Composes system components for arrangement, coordination, and management of cloud resources.

1. **Service Layer:** Defines interfaces for Cloud Consumers to access computing services (SaaS, PaaS, IaaS).
2. **Resource Abstraction and Control Layer:** Provides software abstraction of system components (e.g., hypervisors, virtual machines) and controls for resource allocation, access control, and usage monitoring.
3. **Physical Resource Layer:** Includes all physical computing resources (hardware: CPU, memory, networks, storage) and facility resources (power, cooling, ventilation).

Cloud Service Management All service-related functions for the management and operation of services, covering:

- Business support (e.g., billing, contract management).
- Provisioning/Configuration (e.g., rapid provisioning, resource change).
- Portability/Interoperability (e.g., data portability, VM image migration).

2.4 Cloud Deployment Models

Determines how exclusively cloud resources are made available to cloud consumers (NIST defines 4).

1. **Private cloud:** Solely for use by a single organization (e.g., for enterprises with large-scale IT). Can be on-site or outsourced.
2. **Public cloud:** Available to the general public, shared by all consumers (open market). Concerns include limited SLA, reliability, security, and trust.
3. **Community cloud:** Shared by several organizations supporting a specific community.
4. **Hybrid (federated) cloud:** Two or more public and private clouds that interoperate (e.g., extending private cloud to include a shared public cloud).

3 L03: Cloud Architecture

3.1 How to Organize (Partition) Resources?

3.1.1 1. Workload Distribution Architecture

- **What:** Horizontal IT resource scaling (add more identical resources).
- **How:** A **load balancer** distributes workload to reduce over-utilization or under-utilization of resources (e.g., Amazon Elastic Load Balancing (ELB)).

- **Additional mechanisms:** Audit monitor, cloud usage monitor, hypervisor, logical network perimeter, resource cluster, resource replication.

3.1.2 2. Resource Pooling Architecture

- **Goal:** Aggregate cloud resources of different types to serve diverse needs of cloud consumers and automatically maintain synchronized identical IT resources.
- **Common pools:** Physical server pool, Virtual server pool, CPU pool, Memory pool, Storage pool, Network pool.
- **Dedicated pools:** Group each type of resource into a larger pool, or sub-pool (e.g., a dedicated pool with CPU, memory, storage, and network sub-pools).
- **Complex pools (Hierarchical Structure):**
 - **Sibling resource pool:** Drawn from physically grouped, isolated IT resources not spread across datacenters (for multiple distinct tenants).
 - **Nested resource pool:** Larger pools divided into smaller pools of the same IT resource type (for different departments/groups within the same organization).

3.1.3 3. Dynamic Scalability Architecture (Demand Fluctuations)

- **Why:** Enables variable resource utilization to meet usage demand fluctuations.
- **How:** Based on predefined scaling conditions to trigger dynamic allocation of resources from resource pools.
- **Types:**
 - **Dynamic horizontal scaling:** Resource instances scaled dynamically with fluctuating workloads (e.g., resource replication).
 - **Dynamic vertical scaling:** Scale processing capacity of a single IT resource (e.g., increased memory dynamically when a virtual server is overloaded).
 - **Dynamic relocation:** Resource relocated to a host with larger capacity (different resource types).

3.1.4 4. Elastic Resource Capacity Architecture (Resource Availability Fluctuations)

- **What:** Dynamic provisioning of virtual servers.
- **How:** Allocates and reclaims CPUs/RAMs based on runtime monitoring to ensure resources are leveraged before the capacity threshold is reached.
- Uses an **Intelligent Automation Engine** to send scaling requests to the hypervisor or VIM (Virtual Infrastructure Manager).

3.1.5 5. Service Load Balancing Architecture

- A specialized variation of the workload distribution architecture for scaling cloud service implementations.
- Uses redundant cloud service deployments with a load balancer to dynamically distribute workloads.
- **Types of load balancer:**
 - **Independent Load Balancer:** Separate from cloud services/host servers. Intercepts messages and forwards them to replicate virtual servers to achieve horizontal scaling.
 - **Built-in Load Balancer:** Built into a cloud service as part of its application/server environment (a primary server acts as a load balancer and communicates with neighbors).
- **Example:** Amazon Auto Scaling Service combines with ELB to maintain application availability by scaling EC2 capacity up or down automatically.

3.2 How to Operate/Manage Resources to Meet Certain Objectives?

3.2.1 Cloud Bursting Architecture

- **Goal:** Meet higher usage demand by dynamically scaling ("bursting out") from on-premise resources to cloud-based resources.
- Cloud services are pre-deployed and inactive until bursting occurs.
- Releases resources ("bursts in") to on-premise when no longer required (the reverse).
- Relies on an automated scaling listener to redirect requests and resource replication to maintain synchronicity.

3.2.2 Elastic Disk Provisioning Architecture

- **Goal:** Achieve dynamic storage provisioning and charge based on actual usage, as opposed to fixed disk storage allocation charged at the point of allocation.
- Uses a **Thin-provisioning software** on virtual servers for dynamic storage allocation (via hypervisor).
- A **Pay-per-use monitor** tracks and reports usage for granular billing for the exact amount of storage actually used.

4 L04: Resource Hosting & Datacenter

4.1 Resource Hosting

- **On-premise:** Organizations are responsible for deployment, operation, and maintenance of IT resources and Internet connectivity. They have complete control over QoS and safeguarding.
- **Cloud-based:** Uses multiple ISPs, making QoS management difficult. Requires multiple cloud carriers for connectivity and reliability. Security is a concern. Easier to adopt for applications with relaxed bandwidth and latency requirements.
- The role of **Cloud Carriers** (or ISPs) is important for network access between the cloud consumer/provider and the datacenter.

4.1.1 Latency

- The time it takes a packet to travel from one data node to another.
- Highly variable latency with shared nodes. Important when an application requires swift response time.

4.1.2 Bandwidth

- The number of bits transferred per unit time.
- Important when applications require substantial amounts of data transfer.

4.2 Main Components in a Datacenter

1. **Main server hall:** Compute, storage, and networking equipment organized into hot and cold aisles.
2. **Mechanical yard (or central utility building):** Hosts cooling systems (cooling towers, chillers).
3. **Electrical yard:** Houses generators and power distribution centers.

4.2.1 Hardware Components

- **Server Scaling:** Servers ($\rightarrow$ 1U servers) are placed in racks ($\rightarrow$ 42U Server Rack) to form clusters ($\rightarrow$ Multiple Clusters) to make up the datacenter.
- **Machine Rack:** Supports servers, storage, and networking equipment. Includes power conversion/delivery and battery backup. Uses rack-level switches (e.g., TOR - Top of Rack).

- **Computing Accelerators (e.g., GPUs):** Connected to the CPU host via PCIe-attached accelerator trays with multiple GPUs.
- **Storage:** Data is categorized as private (local DRAM or disk) or part of shared state (distributed storage network).
- **Network:**
 - *Carrier interconnection:* Between external WAN and datacenter LAN.
 - *LAN fabric:* Within a rack, often using Ethernet switch (e.g., TOR).
 - *SAN fabric:* Between servers and storage systems (Fibre channel).

4.2.2 Infrastructure and Protection

- **Building:** Described by floor area or critical power supplied to IT equipment (e.g., multi-story datacenters up to 100 MW critical power).
- **Power, UPS, and Backup:** Datacenters are connected to a power supply with UPS (uninterruptible power supply) for battery backup, and generators for long-term backup.
- **Cooling:**
 - Hot-Cold Aisles for air circulation. CRAC (Computer Room Air Conditioning) units.
 - Water-cooled chillers and cooling towers.
 - Trend: From air to liquid (water) cooling, or using seawater, due to water being $1000\times$ more efficient at removing heat.
- **Fire Protection:** Uses agents like FM200 (Heptafluoropropane extinguishing agent HFC-227ea).
- **Security & Monitoring:** Includes access control and environmental monitoring.

4.3 Datacenter Availability and Efficiency

4.3.1 Datacenter Tiers (Increasing Availability & Cost)

- **Tier I (Basic Capacity):** Single path for power/cooling distribution, no redundant components. Availability $\approx 99.671\%$ (29 hrs downtime/yr).
- **Tier II (Redundant Components):** Adds redundant components (N+1). Availability $\approx 99.741\%$ (23 hrs downtime/yr).

- **Tier III (Concurrently Maintainable):** One active and one alternate distribution path for utilities, both with redundant components. Redundancy even during maintenance. Availability $\approx 99.982\%$ (1.5 hrs downtime/yr).
- **Tier IV (Fault Tolerant):** Two simultaneously active power and cooling distribution paths. Redundant components in each path. Tolerates any single equipment failure. Availability $\approx 99.995\%$ (0.5 hrs downtime/yr).

4.3.2 Energy Usage and Power Efficiency

- **PUE (Power Usage Efficiency):**

$$\text{PUE} = \frac{\text{total datacenter power}}{\text{power used by IT equipment}}$$

- **Ideal PUE** is 1.0. Average US datacenter PUE is ≈ 2.0 . Energy efficient datacenter PUE is ≈ 1.2 .
- **Energy-proportional systems (Ideal):** An energy-proportional system consumes no power when idle, very little power under a light load, and gradually more power as the load increases.
- **Dynamic power range:** The difference between the low and upper range of a device's power consumption. A wider range is better for efficiency.

5 L05: Virtualization & Multitenancy

5.1 Virtualization

- **What:** Enables a single physical infrastructure to function as multiple logical infrastructures or resources.
- Breaks the $1 \rightarrow 1$ relationship between hardware and OS. Many OSs can run and share a single hardware ($1 \rightarrow N$ improves utilization).
- **Why:** Improve resource utilization through sharing (lower cost, increased ROI), improve scalability, ease of administration.

5.1.1 Pros and Cons

- **Pros:**
 - Hardware independence (eliminates software-hardware dependencies).
 - Resource consolidation (different virtual resources share one physical resource $\rightarrow$ increase utilization, load balancing).

- Resource replication (virtual disk images allow agility in migration and rapid scaling).
- **Cons:** Performance overhead, single point of failure of virtualization software.

5.2 Types of Virtualization

1. **Processor virtualization:** Physical multicore shared processor pool mapped to virtual processor pools and virtual machines.
2. **Memory virtualization:** Physical shared distributed main memory pool mapped to virtual main memory pools and virtual machines.
3. **Storage virtualization:** Physical shared disk storage pool mapped to virtual disk storage pools and virtual machines.
4. **Network virtualization:** Uses a virtualization layer to abstract the physical network into virtual private networks.
5. **Data virtualization:** Data virtualization layer relieves applications from knowing the type and location of shared data.
6. **Application virtualization:** Application virtualization layer abstracts shared applications into virtual application instances for users (key enabler for SaaS/PaaS).

5.3 Approaches to Virtualization (Hypervisors)

- **Hypervisor (or Virtual Machine Manager - VMM):** Software that sits between VMs and physical infrastructure to manage VMs and execute the consumer's guest OS.
- Approaches aim to restrict misuse by untrusted user-level programs using **protection rings** (Ring 0 → most privileged, Ring 3 → least privileged).

5.3.1 Virtualization Approaches

1. **Full Virtualization:**
 - *Mechanism:* VMM at Ring 0. Guest OS is **unmodified** and unaware it is virtualized. Uses binary translation to translate guest OS instructions for the virtual infrastructure.
 - *Pros:* Best isolation/security, different OSs run simultaneously, easy migration to native hardware, easy to install.
 - *Cons:* Binary translation overhead (reduces performance), requires compatible hardware/software combination.
2. **Para Virtualization:**

- *Mechanism:* VMM at Ring 0. Guest OS is **modified** (OS-assisted virtualization) and aware it is virtualized. Replaces OS requests with **hypercalls** to communicate directly with the hypervisor without translation.
- *Pros:* Eliminates binary translation overhead (improves performance).
- *Cons:* Overhead of kernel modifications, modified OS cannot migrate to run on physical hardware, lacks backward compatibility.

3. Hardware-assisted Virtualization:

- *Mechanism:* Hardware (processor) extensions support virtualization. OS requests directly trap the hypervisor (eliminates binary translation and OS modification). VMM has the highest root privilege.

5.3.2 Types of Hypervisors

1. **Bare Metal (Type 1):** Runs directly on the physical infrastructure **without** a host OS. Used for most cloud environments.
2. **Hosted (Type 2):** Requires the help of a host OS to communicate with the underlying infrastructure.

5.4 Containers (vs. VMs)

- **Containers** use hosted virtualization, creating multiple isolated user space instances.
- Unlike VMs, containers **share the host system's kernel**, making them lightweight.
- **VMs:** Each VM has its own Guest OS, Bins/Libs, and App in the user space, all running on a Hypervisor over the Server/Host OS.
- **Containers:** Multiple Apps/Bins/Libs run in separate user spaces that **share the Host Kernel** and sit on a Container Engine over the Host OS/Server.

5.5 Multitenancy

- **Cloud Providers:** A single instance of software runs on a server and serves multiple tenants (e.g., cloud platforms, cloud services).
- **Cloud Consumers:** Get a dedicated instance of software with a customized view. Tenants isolation ensures they do not have access to data and configuration they do not own.
- **Multitenancy vs. Virtualization:** Multitenancy focuses on dedicating a consumer instance of software, while virtualization is the abstraction of physical resources.

5.5.1 Characteristics of Multitenancy

- Usage/tenant isolation
- Data security
- Recovery (backup, restore)
- Scalability (increase in usage/number of tenants)
- Metered usage (charged only for features used)
- Data tier isolation (databases, tables, schemas can be isolated or shared)

6 L06: Applications and Paradigms

6.1 Cloud Applications

6.1.1 Common Features of Cloud Providers

- Provides basic and higher-level services (IaaS $\rightarrow$ SaaS).
- Compute and storage resources (virtual servers, object store).
- Deployment management services (load balancer, auto-scaling, monitoring).
- User interface (GUI, command-line interface).

6.1.2 Ideal Applications for Cloud

Applications whose workload can be partitioned into n segments to spawn n instances, reducing execution time close to a factor of n .

- Web services, database services, search services.
- Machine learning with massive-scale models.

Unlikely to Perform Well:

- Applications with a complex workflow and multiple dependencies (e.g., high-performance computing).
- Applications with intensive communication among concurrent instances.
- Workload that cannot be arbitrarily partitioned.

6.1.3 Challenges in Developing Cloud Applications

- **Performance isolation:** Ensuring one customer's performance is not affected by other users sharing the resources.

- **Reliability:** Server failures are expected; mitigation methods like checkpoint/restart are necessary.
- **Network fluctuations:** Latency and bandwidth variations affect performance.
- **Data logging limits:** Performance constraints limit the amount of data logging possible for diagnostics.

6.1.4 Architectural Styles for Cloud Applications

Cloud services use web technology as both the implementation medium and management interface, relying on the client-server architecture.

3-Tier Web Applications Architecture

1. **Presentation layer:** User-interface (client and server side).
2. **Application layer:** Implementation logic (web server handles requests, retrieves resources or generates content).
3. **Data layer:** Persistent data stores (web servers interact with application servers and databases).

Web Services Technology Provides the "as-a-Service" cloud delivery models and a standard interface for communication.

- **SOAP (Simple Object Access Protocol):** Application protocol defining a common web service messaging format (XML-based).
- **REST (Representational State Transfer):** Software architecture for distributed hypermedia systems. Supports client communication with stateless servers, platform/language independent, supports data caching, and uses HTTP with formats like XML and JSON.

6.2 Cloud Application Development Models

(See L02 for basic models. L06 provides examples of setting up a blog using each model.)

- **IaaS (Blog Example):** Creates a compute instance (VM), installs and configures LAMP stack (Linux, Apache, MySQL, PHP), installs/configures Wordpress, configures a new blog, and blogs. (Highest flexibility, most effort).
- **PaaS (Blog Example):** Accesses a system with a ready-made LAMP environment, installs/configures Wordpress, sets up a new blog, and blogs.
- **SaaS (Blog Example):** Accesses a server with Wordpress already installed/configured, sets up a new blog, and blogs. (Lowest flexibility, least effort/fastest time-to-market).

6.3 Function-as-a-Service (FaaS) and Serverless Computing

- **What:** Build scalable event-driven applications (e.g., AWS Lambda).
- **Why:** Relieves cloud consumer server management (provisioning, scaling, monitoring); no idle capacity; developer focuses on application/business logic.
- **How:**
 - The application is divided into smaller, single-purpose **Lambda functions**.
 - An **event** triggers a Lambda function, which dies after execution (Lambda is stateless).
 - The developer provides the code and configuration (event source), while the system manages the infrastructure.
- **Serverless FaaS:** Consumer pays only when code is executing; no virtual machines/containers are exposed in the programming model; vendor provides provision-free scalability.

6.4 MapReduce Programming Model

- **Goal:** Supports arbitrarily divisible workload and distributed computing on large data sets (e.g., web-scale data, 100s of GBs to PBs).
- **Key idea:**
 - **Split** input data into blocks and assign each block to an instance/process for parallel execution.
 - **Merge** partial results produced by individual instances after execution.
- Transforms a set of input $\langle key, value \rangle$ pairs into a set of output $\langle key, value \rangle$ pairs.
- Programmers implement the two main functions:
 - **MAP:** Extracts data you need from each record (e.g., given a log, outputs $\langle URL, 1 \rangle$ for each request).
 - **REDUCE:** Aggregates, summarizes, filters, or transforms the extracted data (e.g., outputs $\langle URL, total\ count \rangle$).

6.4.1 Hadoop

- **Hadoop MapReduce:** An open-source implementation of MapReduce (developed by Apache + Yahoo, Java-based).

- Presents MapReduce as an analytics engine with a distributed storage layer called **Hadoop Distributed File System (HDFS)**, which mimics Google File System (GFS).

7 L08: Cloud Software Development

7.1 SaaS vs. Traditional Software

- **SaaS Changes How Software is Delivered:** usage-based billing, high scalability, ease of access, automated updates.
- **PaaS Changes How (SaaS) Software is Developed:** Automates deployment, testing, and scaling, reducing manual work and cost.

7.1.1 SaaS is Different

- **Pay-per-use:** Web access to commercial software on a pay-as-you-use basis.
- **Zero infrastructure:** Customers need not install the software (SaaS is developed, deployed, and managed by the provider).
- **Reduced business cost:** Multi-tenancy (1-to-many relationship) vs. traditional (1-to-1).
- **Automated updates:** Performed by the SaaS provider, not the user.

7.1.2 Suitability of SaaS

- **Suitable:** On-demand software required, start-up company (cannot invest in licenses), applications with unpredictable/dynamic load.
- **Not suitable:** Real-time processing needed, highly confidential data (data localization required), on-premise application fulfills needs.

7.2 Different Perspectives of SaaS Development

The key challenges are choosing the correct multitenancy level(s) and managing governance/security.

1. SaaS with Self-managed Infrastructure and Self-managed Platform

- Everything self-managed, not cloud-enabled.
- Multitenancy: SaaS layer only (shared application).
- Full control (secure data), but higher cost and lower resource utilization.

2. SaaS with Self-managed Platform (on shared IaaS)

- Shared SaaS application on shared IaaS, but self-managed platform.
- Multitenancy: SaaS & IaaS layers.
- Higher infrastructure resource utilization, but overheads of maintaining the platform.

3. SaaS with Self-managed Infrastructure (on shared PaaS)

- Shared SaaS application on shared PaaS, deployed on on-premise infrastructure.
- Multitenancy: SaaS & PaaS layers.
- Full governance over user data (e.g., Community cloud deployment).

4. SaaS with Cloud-enabled IaaS and PaaS

- Shared everything.
- Multitenancy: at all layers.
- Highest scalability/availability (via dynamic scaling, backup/recovery by providers).
- Con: No governance over user data, risk of cross-tenant attacks.

7.3 Cloud-aware Software Development using PaaS

PaaS tools allow developers to develop SaaS applications online and deploy them on the service provider's infrastructure.

- **Requirement Analysis:** Determine delivery model (security, ROI) and deployment model (Public PaaS for low maintenance, Private PaaS for moderate security/governance).
- **Multitenant Architecture:** Trade-off between resource utilization (higher multitenancy level) and data security (lower multitenancy level).
- **Dynamic Scaling and Availability:**
 - *Dynamic scaling:* Use software/hardware load balancers to distribute requests and scale computing power.
 - *High availability:* Use multiple replica copies of virtual machines and database applications.
 - *Recovery time:* Replica should be near customer location.
- **Database Design (Database-level Multitenancy):** Options include Separate Database, Shared Database & Separate Schema, or Shared (everything) Database & Shared Schema. The choice affects application performance, scalability, and availability. NoSQL databases may be a better option for diverse data types than traditional relational databases.

- **SaaS Development using PaaS:** SaaS should include a good UI design, role-based access control, performance monitoring tools, control panels, self-service sign-up, and usage statistics/billing.
- **Monitoring and SLA Maintenance:** Monitor for tenant misbehavior, failure, security attacks, and SLA violations. Frequent automated updates are necessary to provide the latest version.

8 L09: Pricing Models and Modeling TCO

8.1 Pricing Models

Define the unit cost of resource usage according to usage cost metrics.

8.1.1 Factors Influencing Price Model

1. Overhead in design, development, deployment, and operation.
2. Opportunities to reduce expenses (resource sharing, data center optimization).
3. Market competition and regulatory requirements.

8.1.2 Key Variables

- **Cost metrics and prices:** Based on on-demand (pay-per-use) or reserved allocation (upfront fee + discounted hourly rate).
- **Fixed (usage quota) and variable (actual usage) rates.**
- **Discounts** for higher volumes.
- **Payment options/schedules:** Prepaid, postpaid, installments.

8.1.3 Examples of Pricing

- **IaaS:** Based on resource allocation/usage (network data transferred, number of virtual servers, storage capacity).
- **PaaS:** Also depends on software configurations, development tools, and licensing fees.
- **SaaS:** Determined by application modules, nominated users, number of transactions.

8.1.4 SaaS Pricing Models

- **Integrated Pricing Model:** Cloud Consumer leases a SaaS product from Provider A, and Provider A leases the IaaS from Provider B to host the SaaS. Consumer pays only Provider A, and Provider A pays Provider B.

- **Separate Pricing Model:** Cloud Consumer leases IaaS from Provider B and SaaS from Provider A to host the SaaS. Consumer pays both Provider A and B.

8.1.5 AWS Pricing Options (EC2)

- **On-demand:** Multi-tenancy, pay by the second, start/stop as you wish (good for low, unpredictable spikes).
- **Reserved:** Pay by the year (upfront + discounted hourly rate), fixed duration (1 or 3 years), stop/start as needed (good for constant, predictable capacity).
- **Spot:** Bid for unused capacity; instance terminates if the spot price is less than the current market rate (good for capacity needs that can be interrupted).
- **Dedicated:** Instance runs on hardware dedicated to a single customer (good for strict regulatory requirements, not shared with other AWS accounts).

8.2 Cost Metrics

8.2.1 Business Cost Metrics

- **Up-front costs (Capital Expenses - Capex):** Initial investment for IT resources (hardware, software, deployment). High for on-premise, low for cloud-based.
- **On-going costs (Operational Expenses - Opex):** Recurring costs to run/maintain IT resources (licensing, electricity, maintenance). Higher for on-premise.
- **Additional Costs:**
 - *Cost of capital:* Cost to raise funds (if high, cloud leasing is justified).
 - *Sunk costs:* Prior investment in existing IT (if significant, cloud transition is harder to justify).
 - *Integration costs:* Effort to inter-operate IT resources with the new environment.
 - *Locked-in costs:* Cost to move from one cloud provider to another.

8.2.2 Cloud Usage Cost Metrics

1. **Network usage (data transfer):** Inbound, outbound, and intra-cloud network traffic (often volume-based, sometimes tier-priced).
2. **Server usage (compute):** On-demand or reserved virtual machine allocation (based on performance attributes like CPU, RAM).
3. **Cloud storage device (storage):** On-demand storage allocation size and I/O data transferred.

4. **Cloud service (service - SaaS/PaaS):** Subscription duration, number of nominated users, number of transactions.

8.3 Total Cost of Ownership (TCO) of a Datacenter

- **Costs = Capex + Opex**
- **Capex (Upfront Investment):** Datacenter construction cost, server purchase price (depreciated over time).
- **Opex (Recurring Cost):** Electricity costs, repairs/maintenance, salaries.
- **TCO = (Datacenter depreciation + Datacenter Opex) + (Server depreciation + Server Opex)**